Task 10: Billing Analysis (Data Types)

Student name	Sk.Thohith	Register number	1912260555
GitHub repository	https://github.com/[username]/CSA02-[RegisterNumber]-Assignment1/tree/main/Task10	Compile command	gcc -Wall -Wextra -std=c11 -O2 main.c -o task
Compiler	Online C compiler (screenshots); own benchmarks GCC 13.3.0	Date	26 October 2026

1. Task 10 - Billing Analysis

1.1 Problem analysis

A utility bills three groups of households (4,000, 60,000 and 800,000) at a flat Rs 162.50 per household. The program must compute the total for each group, show it in paise and in rupees, demonstrate the limit of `int`, and show why floating-point is unsuitable for money. The final conclusion printed by the program is: use `long long` for large monetary totals in paise, and use integer paise to avoid floating-point money errors.

1.2 Data types, constants and preprocessor concepts

Item	Type / form	Reason
households[]	int[3] = {4000, 60000, 800000}	Counts fit in 32 bits
bill_per_household	long long = 16250	Rs 162.50 stored as 16250 paise: exact integer
total	long long = (long long)households[i] * bill_per_household	Cast *before* multiplying so the product is computed in 64 bits
INT_MAX, LLONG_MAX	Macros from <limits.h>	Print the actual limits of this compiler: 2147483647 and 9223372036854775807
%lld, %d, %.17f	printf conversion specifiers	Specifier must match the argument type or output is undefined
total / 100, total % 100	Integer division and remainder	Splits paise into rupees and paise; %02lld pads paise to two digits

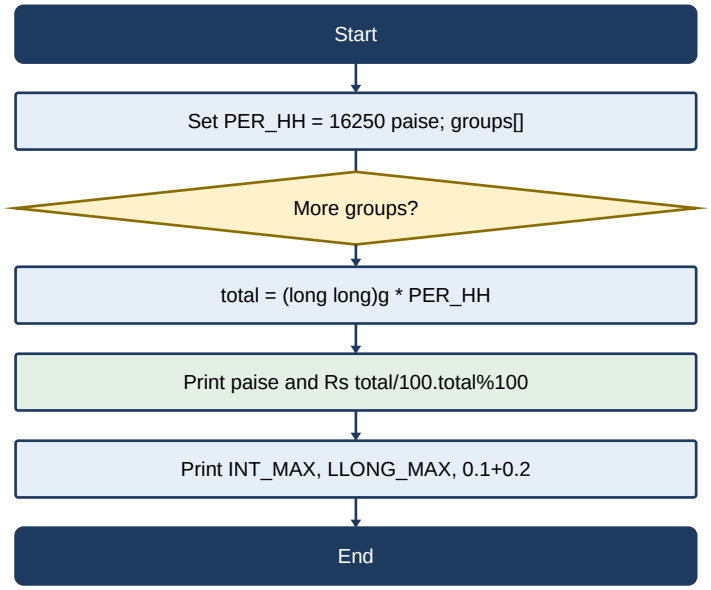
1.3 Source code (visible lines 7-43 of main.c)

```
int households[] = {4000, 60000, 800000};
long long bill_per_household = 16250; /* Rs 162.50 in paise */
int n = 3;
printf("===== TASK 10: BILLING ANALYSIS =====\n");
printf("Bill per household: Rs 162.50\n\n");
printf("%-15s %-25s %-20s\n", "Households", "Total Bill (Paise)", "Total Bill (Rupees)");
for (int i = 0; i < n; i++) {
    long long total = (long long)households[i] * bill_per_household;
    printf("%-15d %-25lld Rs %lld.%02lld\n", households[i], total, total / 100, total % 100);
}
printf("Maximum int value       : %d\n", INT_MAX);
printf("Maximum long long value : %lld\n", LLONG_MAX);
double amount = 0.1 + 0.2;
printf("0.1 + 0.2 = %.17f\n", amount);
/* ...conclusion printf lines, return 0; */
```

1.4 Pseudocode

```
PER_HH  <- 16250 paise (long long)
FOR each group g in {4000, 60000, 800000}
    total <- (long long)g * PER_HH
    PRINT g, total, total DIV 100 "." (total MOD 100, 2 digits)
PRINT INT_MAX, LLONG_MAX
PRINT 0.1 + 0.2 with 17 decimals
```

UML activity diagram - Task 10



1.5 Output evidence

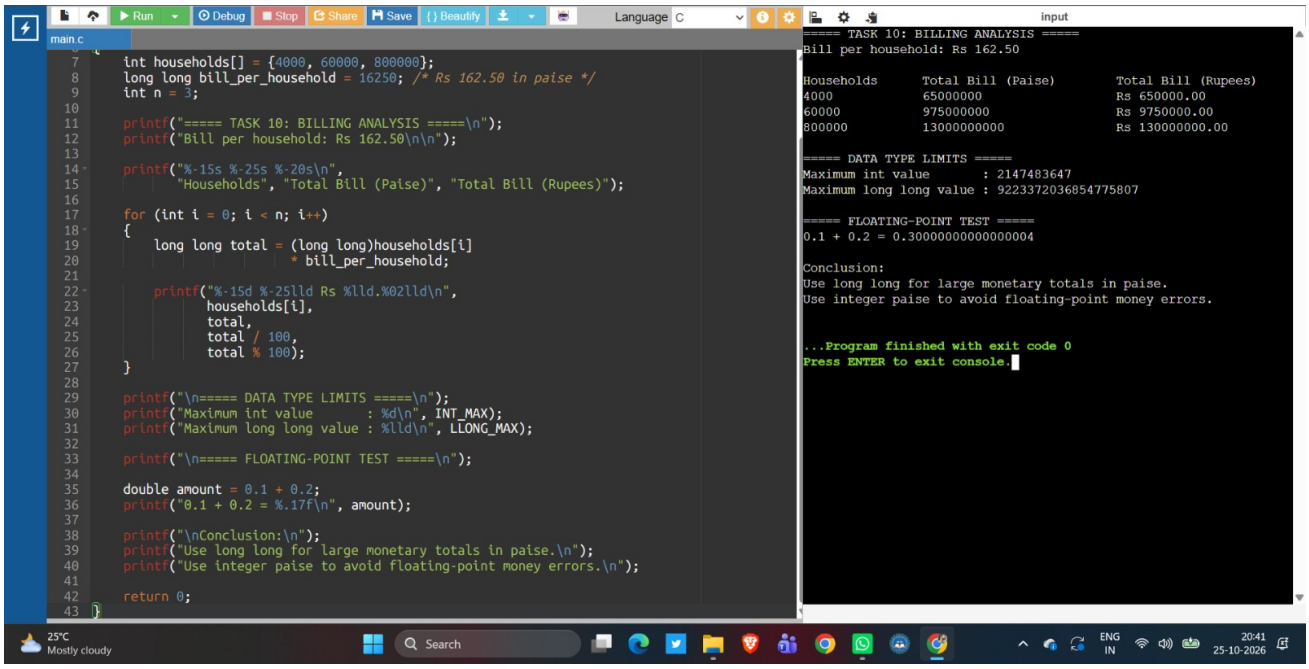


Figure 1 - Task 10 source and console output.

1.6 Test table (expected vs actual)

#	Case	Expected (hand calculation)	Actual (screenshot)	Result
1	4000 x 16250	65,000,000 paise = Rs 650000.00	65000000 / Rs 650000.00	Pass
2	60000 x 16250	975,000,000 paise = Rs 9750000.00	975000000 / Rs 9750000.00	Pass
3	800000 x 16250 (boundary: exceeds int)	13,000,000,000 paise = Rs 130000000.00	13000000000 / Rs 130000000.00	Pass
4	INT_MAX	2147483647 (32-bit int)	2147483647	Pass
5	LLONG_MAX	9223372036854775807	9223372036854775807	Pass
6	0.1 + 0.2 as double	Not exactly 0.3 (binary representation)	0.30000000000000004	Pass (error demonstrated)

Why the cast matters. Without `(long long)`, `800000 * 16250` would be evaluated in 32-bit int. I computed the wrapped value on GCC 13.3.0: it becomes **115098112** instead of 13000000000, so the bill would be silently wrong by about Rs 129 million. The largest total that fits in a signed 64-bit value is 9223372036854775807 / 16250 = 567,592,125,344,909 households, far beyond any real population.

1.7 Alternative concept: floating-point rupees (double) instead of integer paise

If the bill were stored as `double bill = 162.50`, 162.50 happens to be exactly representable in binary (162 + 1/2), so these three totals would still print correctly. The danger appears with amounts such as 0.10 or 0.20, which are not exactly representable, as the program itself shows (0.1 + 0.2 = 0.30000000000000004). Repeated additions drift, and equality comparisons such as `total == 650000.00` can fail.

```
double total = households[i] * 162.50;      /* alternative */
printf("Rs %.2f\n", total);                  /* rounds only when printing */
```

	Integer paise (long long)	Floating-point rupees (double)
Time complexity	O(n) for n groups; O(1) per group	O(n); O(1) per group (same asymptotically)
Space complexity	O(1) extra (8 bytes per amount)	O(1) extra (8 bytes per amount)
Exactness	Exact while below 9.22e18 paise	About 15-17 significant digits; sums can drift
Range	+/- 9.22e18 paise (about Rs 9.2e16)	Huge range but not exact for cents
Verdict	Recommended for money	Acceptable for statistics, not for ledgers

Big-O is identical because both do one multiplication per group; the difference is correctness, not asymptotic cost. **My measurement** (GCC 13.3.0 -O2, sandbox machine, 10^8 iterations of a billing kernel): integer path 0.223 s, double path 0.157 s. The double version was about 30% faster in this micro-benchmark, which is outweighed by exactness when handling money. Timings depend on the machine and compiler; re-run on yours for your own table.